

TravelPlan

1. Raffinamento dei requisiti	2
Versione 1 “semplice”	4
2. Diagramma UML delle Classi	4
3. Specifica dei tipi di dato	4
4. Specifica delle operazioni di classe	5
Operazioni ausiliarie	5
non_sovraposti(iniz1, fin1, iniz2, fin2) : Boolean	5
in_periodo(iniz1, fin1, iniz2, fin2) : Boolean	5
Specifica della classe Utente	5
[V.Utente.partecipa.AttivitadiViaggio]	5
Specifica della classe Viaggio	5
Vincoli esterni	5
[V.Viaggio.luogoDestinazione.Uguale.ProssimoLuogoAttivita]	5
[V.Viaggio.minBudget]	6
[V.Viaggio.MinMaxPartecipanti]	6
[V.Viaggio.LimitPartecipanti]	6
attivit_max_partecipanti() : Attivita	6
Specifica della classe Attivita	6
Vincoli esterni	6
[V.Attivita.NonSovraposti]	6
[V.Attivita.FineDopoInizio]	6
fine_attivita() : DataOra	7
partecipanti() : Utente[0..*]	7
Specifica della classe AttivitaComposta	7
Vincoli esterni	7
[V.AttivitaComposta.durata]	7
[V.AttivitaComposta.inclusione.AttivitaSemplici]	7
durata(): Reale > 0	7
fine_attivita(): DataOra	8
partecipanti() : Utente[0..*]	8
5. Diagramma UML degli Use-Case	8
6. Specifica delle operazioni di Use-Case	9
Specifiche di operazioni di use-case Statistiche	9
num_spostamenti_in_viaggio(u:Utente, v: Viaggio): Intero >= 0	9
utenti_partecip_max_num_attivita_in_viaggio(v: Viaggio): Utente [1..*]	9
citta_con_max_viaggi(naz: Nazione): (Regione, Citta) [1..*]	9
num_viaggi_nel_mese_anno_precedente(cit: Citta): (mese: intero in [1..12], intero >= 0) [12]	10
Versione 2 “complicata”	11
1. Diagramma UML delle Classi	11
2. Specifica dei tipi di dato	12
3. Specifica delle operazioni di classe	12
Operazioni ausiliarie	12
non_sovraposti(iniz1, fin1, iniz2, fin2) : Boolean	12
in_periodo(iniz1, fin1, iniz2, fin2) : Boolean	12
Specifica della classe Utente	12
[V.Utente.partecipa.AttivitadiViaggio]	12
viaggi_partecipa() : Viaggio[0..*]	13
attivit_in_viaggio(v: Viaggio) : Attivita [0..*]	13
Specifica della classe Viaggio	13

[V.Viaggio.luogoDestinazione.Uguale.ProssimoLuogoAttivita]	13
[V.Viaggio.AttivitaNonSovrapostiNelViaggio]	13
[V.Viaggio.minBudget]	13
[V.Viaggio.MinMaxPartecipanti]	14
[V.Viaggio.LimitPartecipanti]	14
attivita_max_partecipanti() : Attivita	14
citta_toccate() : Citta [0..*]	14
Specifica della classe Attivita	14
Operazioni ausiliarie	14
ha_figlie(): Boolean	14
Vincoli esterni	15
[V.Attivita.FineDopoInizio]	15
[V.Attivita.CampiPermessi]	15
durata_minuti() : Intero > 0	15
fine_attivita() : DataOra	15
partecipanti() : Utente[0..*]	16
Specifica della classe AttivitaSemplice	16
Vincoli esterni	16
[V.AttivitaSemplice.SenzaFiglie]	16
Specifica della classe Spostamento	16
Vincoli esterni	16
[V.Spostamento.LuoghiDiversi]	16
citta_partenza(): Citta	16
citta_arrivo(): Citta	16
Specifica della classe AttivitaComposta	17
Vincoli esterni	17
[V.AttivitaComposta.HaFiglie]	17
[V.AttivitaComposta.IncludeFigliDiretti]	17
[V.AttivitaComposta.noCicli]	17
1.5. Diagramma UML degli Use-Case	17
1.6. Specifica delle operazioni di Use-Case	17
Specifiche di operazioni di use-case Statistiche	17
num_spostamenti_in_viaggio(u:Utente, v: Viaggio): Intero >= 0	17
utenti_partecip_max_num_attivita_in_viaggio(v: Viaggio): Utente [1..*]	18
citta_con_max_viaggi(naz: Nazione): (Regione, Citta) [1..*]	18
num_viaggi_nel_mese_anno_precedente(cit: Citta): (mese: intero in [1..12], intero >= 0) [12]	19

1. Raffinamento dei requisiti

1. Requisiti relative agli utenti
 - 1.1. nome
 - 1.2. cognome
 - 1.3. email
 - 1.4. data iscrizione
 - 1.5. citta di provenienza
 - 1.6. viaggi in quali partecipano
 - 1.6.1. attività alle quali partecipano nei viaggi
2. Requisiti relative di viaggi
 - 2.1. nome
 - 2.2. numero minimo di partecipanti

- 2.3. numero massimo di partecipanti
- 2.4. attività ai quali è associato
- 2.5. budget
- 3. Requisiti relativi alle attività
 - 3.1. nome
 - 3.2. istante inizio
 - 3.3. durata
 - 3.4. prezzo
 - 3.5. luogo in qui si tiene
 - 3.6. descrizione
 - 3.7. tipo attività
 - 3.7.1. visite
 - 3.7.2. pasti
 - 3.7.3. tour
 - 3.7.4. trasporti
 - 3.7.5. pernottamenti
 - 3.7.6. spostamenti
 - 3.7.6.1. luogo di partenza
 - 3.7.6.2. luogo di arrivo
 - 3.8. tipo struttura
 - 3.8.1. semplici
 - 3.8.2. composte (raggruppano più attività semplici)
 - 3.9. utenti che partecipano
- 4. Requisiti relativi ai luoghi
 - 4.1. indirizzo
 - 4.2. città
 - 4.3. regione
 - 4.4. nazione

999.1 Sistema deve permettere agli utenti registrati di:

- partecipare a viaggi
- creare i viaggi in qualità di organizzatori, aggiungendo attività di vario tipo
- trovare i viaggi che includono una certa destinazione in certo intervallo di tempo
- trovare le città toccate dal maggior numero di viaggi in un certo intervallo di tempo
- dato:
 - un budget_min e budget_max,
 - un insieme dei regioni,
 - un periodo di tempo,
 - un numero N

calcolare insieme dei viaggi che hanno:

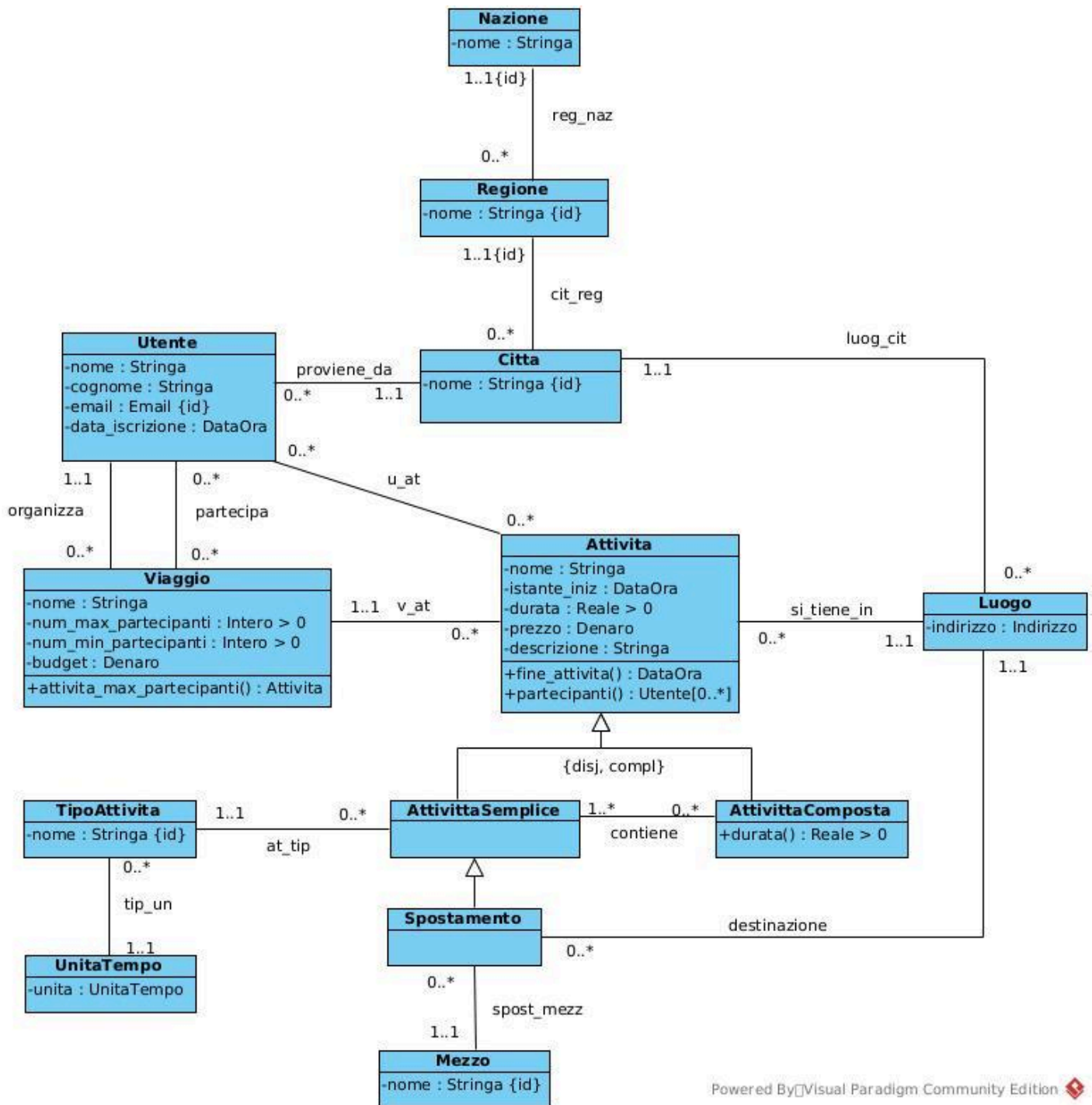
- un badget nel intervallo richiesto,
- toccano almeno una delle regioni date,
- si svolgono interamente nel periodo dato,
- sono organizzati da un utente che abbia organizzato almeno N viaggi in precedenza.

999.2. Sistema permette ai responsabili del sistema di

- calcolare, per ogni regione di una data nazione, il numero di viaggi organizzate in un dato periodo di tempo che toccano tale regione
- calcolare, data una città di destinazione, per ognuno dei 12 mesi dell'anno, il numero di viaggi organizzati in quel mese nell'ultimo anno solare

Versione 1 “semplice”

2. Diagramma UML delle Classi



3. Specifica dei tipi di dato

Tipo **UnitàTempo** = {minuti, ore, giorni, settimane, mesi}

Tipo **Email**: Stringa secondo lo standard

Tipo **Denaro**: Tipo record con campi

- importo: Reale ≥ 0
- valuta: Stringa di 3 caratteri alfabetici maiuscoli

Tipo **Indirizzo**: Tipo record con campi

- via: Stringa

- civico: Stringa
- CAP: Stringa di 5 caratteri numerici

4. Specifica delle operazioni di classe

Operazioni ausiliarie

non_sovraposti(iniz1, fin1, iniz2, fin2) : Boolean

precondizioni:

- $iniz1 < fin1$ e $iniz2 < fin2$

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- $result = True$
se e solo se
 $fin1 \leqslant iniz2$ or $fin2 \leqslant iniz1$

in_periodo(iniz1, fin1, iniz2, fin2) : Boolean

precondizioni:

- $iniz1 < fin1$ e $iniz2 < fin2$

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- $result = True$
se e solo se
 $iniz1 \leqslant iniz2$ and $fin2 \leqslant fin1$

Specifica della classe Utente

[V.Utente.partecipa.AttivitadiViaggio]

Per ogni u: Utente tale che esiste

- $(u, at): u_at, at: Attivita$ e $(v, at): v_at, v: Viaggio$

deve esistere

- $(u, v): partecipa$

Specifica della classe Viaggio

Vincoli esterni

[V.Viaggio.luogoDestinazione.Uguale.ProssimoLuogoAttivita]

per ogni v: Viaggio

- sia A l'insieme di tutti at: Attivita tali che $(v, at): v_at$
- per ogni copia sp1 e at2 in A tali che
 - sp1: Spostamento,
 - $sp1.fine_attivita \leqslant at2.istante_inizio$
 - non esiste nessun sp2: Spostamento in A tale che:
 - $sp1.fine_attivita < sp2.istante_inizio$
 - $sp2.fine_attivita < at2.istante_inizio$

○ sia l1: Luogo tale che $(sp1, l1): destinazione$

○ sia l2: Luogo tale che $(at2, l2): si_tiene_in$

deve essere vero che:

- $l1 = l2$

[V.Viaggio.minBudget]

per ogni v: Viaggio

- sia A l'insieme di tutti at: Attivita tali che (v, at): v_at
- sia sum è la somma di tutti at.prezzo.importo con at.prezzo.valuta uguale per at in A

deve essere vero che:

v.budget >= sum

[V.Viaggio.MinMaxPartecipanti]

per ogni v: Viaggio deve essere vero che:

- v.num_min_partecipanti <= v.num_max_partecipanti

[V.Viaggio.LimitPartecipanti]

per ogni v: Viaggio

- sia U insieme di u:Utenti tali che (u, v): partecipa

deve essere vero che:

- v.num_min_partecipanti <= |N| <= v.num_max_partecipanti

attivit _max_partecipanti() : Attivita

precondizioni:

- esiste almeno un link (this, at): v_at, at:Attivita

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- il risultato "result" viene definito come segue:
 - sia A insieme dei at:Attivita tali che (this, at): v_at
 - sia max è il valore massimo di at.partecipanti() per at in A
 - sia at_max: Attivita in A tale che at_max.partecipanti() = max

result = at_max

Specifica della classe Attivita

Vincoli esterni

[V.Attivita.NonSovraposti]

per ogni v: Viaggio

- sia A l'insieme di tutti at: Attivita tali che (v, at): v_at
- per ogni at1 e at2 in A deve essere vero che:
non_sovraposti(

at1.istante_iniz,
at1.fine_attivita,
at2.istante_iniz,
at2.fine_attivita) = True

[V.Attivita.FineDopoInizio]

per ogni at: Attivita deve essere vero che at.istante_inizio < at.fine_attivita

fine_attivita() : DataOra

precondizioni: nessuna

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- il risultato “result” viene definito come segue:
 - sia (this, tip): at_tip, tip:TipoAttivita
 - sia (tip, un): tip_un, un: UnitaTempo
 - sia u il valore dell'attributo unita di un
 - Sia mu cosi definito:
 - se u = minuti, mu = 1
 - se u = ore, mu = 60
 - se u = giorni, mu = 1440
 - se u = settimane, mu = 10080

result = this.istante_iniz + (this.durata * mu)

partecipanti() : Utente[0..*]

precondizioni: nessuna

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- il risultato “result” è definito come segue:
 - sia (v, this): v_at, v: Viaggio
 - sia U l'insieme di u:Utente tali che (u, v): partecipa
 - sia P l'insieme di u: Utenti in U tali che (u, this): u_at

result = insieme di u: Utente in P

Specifica della classe AttivitaComposta

Vincoli esterni

[V.AttivitaComposta.durata]

per ogni atc: AttivitaComposta

- sia AS l'insieme di tutti ats: AttivitaSemplici tali che (atc, ats): contiene
- sia sumMinuti la somma di tutti ats.durata di ats in AS

deve essere vero che:

atc.durata = sumMinuti

[V.AttivitaComposta.inclusione.AttivitaSemplici]

per ogni atc: AttivitaComposta

- sia AS l'insieme di tutti ats: AttivitaSemplici tali che (atc, ats): contiene

deve essere vero che

per ogni ats in AS in_periodo(

atc.istante_iniz,
atc.fine_attivita,
ats.istante_iniz,
ats.fine_attivita) = True

durata(): Reale > 0

precondizioni:

(sovrascrive attributo durata ????????)

- esiste almeno un link (this, ats): contiene, ats: AttivitaSemplice

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- il risultato “result” è definito come segue
 - sia AS l'insieme di tutti ats: AttivitaSemplici tali che (this, ats): contiene

- sia sum la somma di tutti ats.durata di ats in AS misurata in uguale UnitaTempo
- result = sum

fine_attivita(): DataOra

precondizioni:

- esiste almeno un link (this, ats): contiene, ats: AttivitaSemplice

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- il risultato "result" è definito come segue
 - sia AS l'insieme di tutti ats: AttivitaSemplice tali che (this, ats): contiene
 - sia max_fine_attivita è il valore massimo di tutti ats.fine_attivita() in AS

result = max_fine_attivita

partecipanti() : Utente[0..*]

precondizioni:

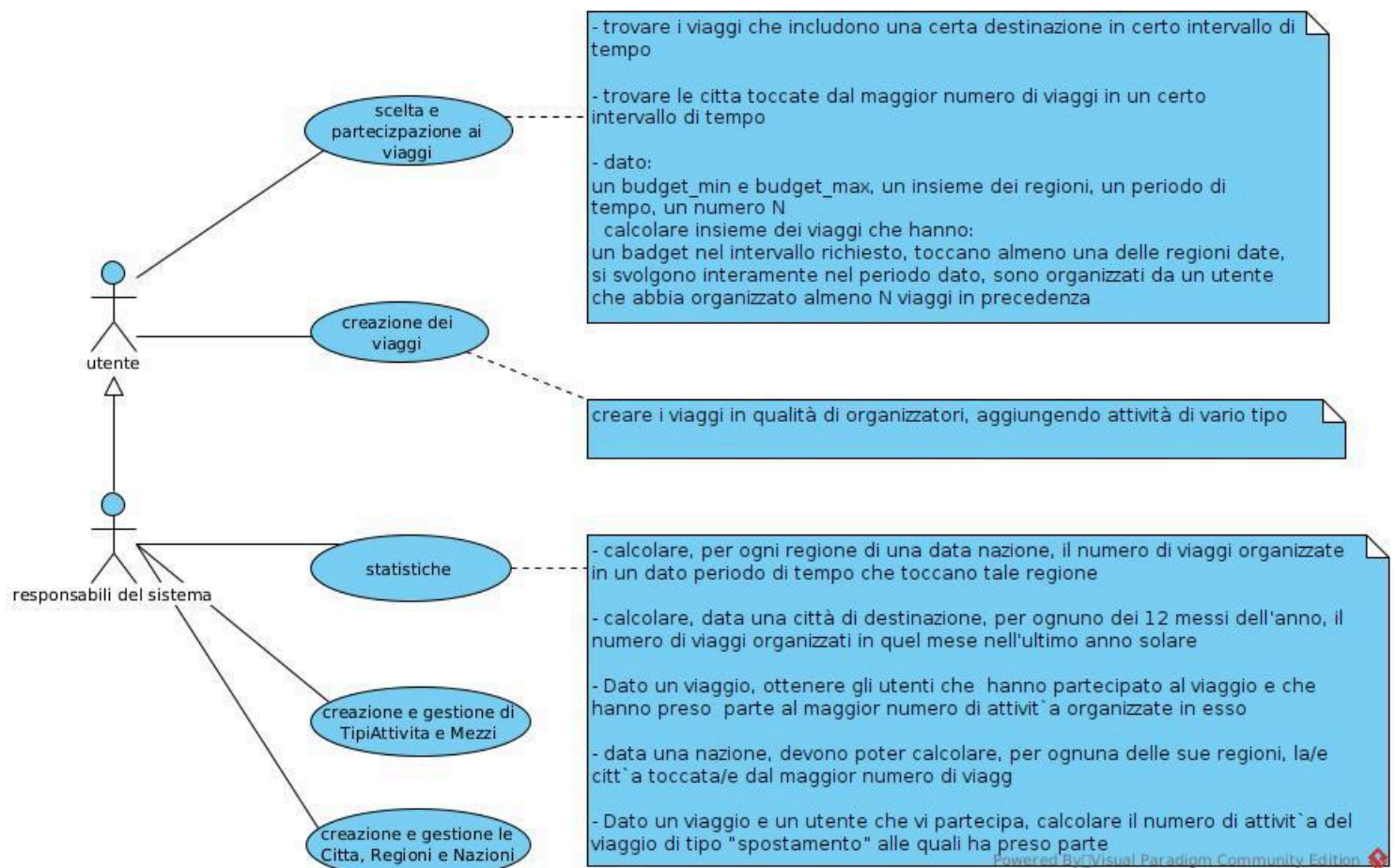
- esiste almeno un link (this, ats): contiene, ats: AttivitaSemplice

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- il risultato "result" è definito come segue:
 - sia AS l'insieme di tutti ats: AttivitaSemplici tali che (this, ats): contiene
 - sia Ut l'insieme di u:Utenti composto da tutti ats.partecipanti() per ogni ats in AS

result = insieme di u: Utente in Ut

5. Diagramma UML degli Use-Case



6. Specifica delle operazioni di Use-Case

Specifiche di operazioni di use-case Statistiche

Dato un viaggio e un utente che vi partecipa, calcolare il numero di attività del viaggio di tipo "spostamento" alle quali ha preso parte.

num_spostamenti_in_viaggio(u:Utente, v: Viaggio): Intero ≥ 0

precondizioni:

- esiste link (u, v): partecipa

postcondizioni:

- operazione non modifica i dati
 - Il risultato "result" è così definito
 - sia S l'insieme dei sp:Spostamenti tali che (u, sp): u_at e (v, sp): v_at
- result = | S |

Dato un viaggio, ottenere gli utenti che hanno partecipato al viaggio e che hanno preso parte al maggior numero di attività organizzate in esso

utenti_partecip_max_num_attivita_in_viaggio(v: Viaggio): Utente [1..*]

precondizioni:

- esiste almeno un link (u, v): partecipa, u:Utente
- esiste almeno un link (v, at): v_at, at: Attività

postcondizioni:

- operazione non modifica i dati
 - Il risultato "result" è così definito
 - sia U_t l'insieme di tutti u: Utenti tali che (u, v): partecipa
 - sia A l'insieme di tutti at: Attività tali che (v, at): v_at
 - sia N insieme delle copie (u, n) di valori tali che
 - u in U_t
 - n = la cardinalità dell'insieme dei link (u, at): u_at, at in A
 - sia n_max il valore massimo di valori n in coppie (u, n) in N
 - sia U_{A_max} l'insieme di tutti u delle copie (u, n) in N, il valore di quali n = n_max
- result insieme di u: Utenti in U_{A_max}

I responsabili del sistema, data una nazione, devono poter calcolare, per ognuna delle sue regioni, la/e città a toccata/e dal maggior numero di viaggi.

citta_con_max_viaggi(naz: Nazione): (Regione, Città) [1..*]

precondizioni:

- deve esistere almeno v:Viaggio tale che (v, at): v_at dove at: Attività tale che
 - esiste un link (at, lu): si_tiene_in , lu: Luogo tale che
 - esiste un link (lu, cit): luog_cit, cit:Città tale che
 - esiste link (cit, reg): cit_reg, reg: Regione tale che
 - esiste un link (reg, naz): reg_naz

postcondizioni:

- operazione non modifica i dati
- il risultato "result" è così definito:
 - sia R insieme delle reg:Regione tale che (reg, naz): reg_naz
 - Sia **M** l'insieme delle coppie (**reg, cit_max**) definite come segue:
 - per ogni reg in R
 - sia C l'insieme di tutte cit: Città tali che (cit, reg): cit_reg
 - sia **NC** insieme delle coppie (**cit, n_cit**) tali che

- per ogni c in C
 - sia V_cit l'insieme di tutti v: Viaggio tali che esistono at: Attività e lu:Luogo tali che
 - (v, at): v_at
 - (at, lu): si_tiene_in
 - (lu, cit): $luog_cit$
 - sia $n_cit = |V_cit|$
- sia n_max il massimo valore n_cit per cit in C
- sia C_max insieme di tutte le cit in C tali che (cit, n_cit) e $n_cit = n_max$
- aggiungi a M tutte le copie (reg, cit) per ogni cit in C_max

result = insieme di coppie (reg, cit_max) in M

aiuto!!!!

Data una città di destinazione, per ognuno dei 12 mesi dell'anno, calcolare il numero di viaggi organizzati in tale mese nell'ultimo anno solare, in tale città

num_viaggi_nel_mese_anno_precedente(cit: Città): (mese: intero in [1..12], intero ≥ 0) [12]

precondizioni:

-
- deve esistere almeno un link (at, lu):luogo_cit , at: Attività, lu: Luogo tale che
 - esiste un link (lu, cit): luogo_cit, cit:Città tale che
 - esiste link (cit, reg): cit_reg, reg: Regione tale che
 - esiste un link (reg, naz): reg_naz

postcondizioni:

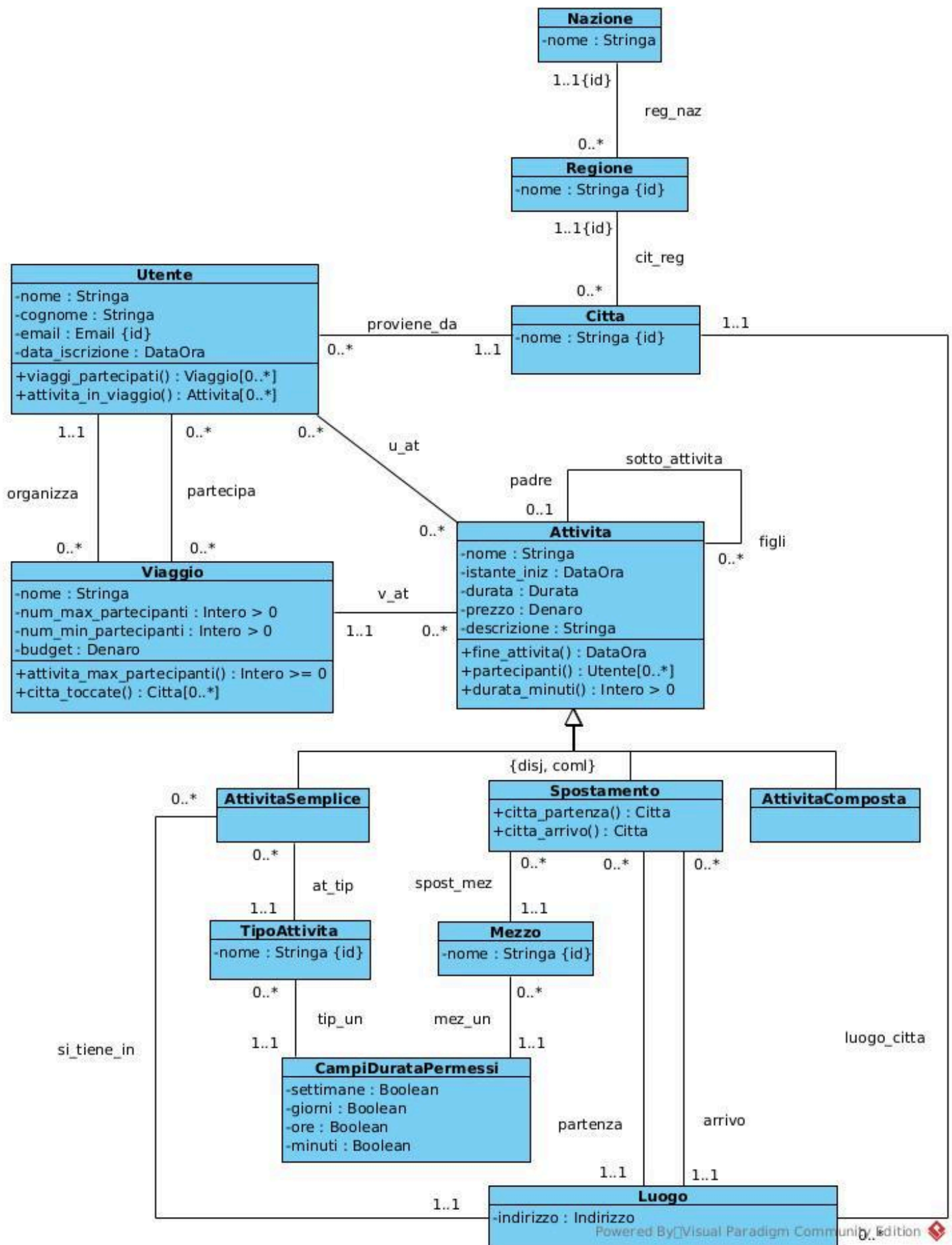
- operazione non modifica i dati
- il risultato "result" è così definito:
 - sia $y = anno(oggi) - 1$ ultimo anno solare
 - sia M l'insieme di m tali che hanno valore intero da 1 a 12
 - sia N l'insieme di coppie (m, n) tali che
 - per ogni m in M
 - V_m l'insieme di tutti v: Viaggi tali che esistono at: Attività, lu: Luogo che
 - (v, at): v_at
 - (at, lu): si_tiene_in
 - (lu, cit): $luog_cit$
 - $anno(at.istante_inizi) = y$
 - $mese(at.istante_inizi) = m$
 - sia V_m_d insieme di tutti v: Viaggi distinti in V_m
 - $n = |V_m_d|$

result = insieme di coppie (m, n) in N

Domanda: posso scrivere $anno(DataOra)$ e $mese(DataOra)$ per prendere anno e mese dalla data che mi interessa?

Versione 2 “complicata”

1. Diagramma UML delle Classi



2. Specifica dei tipi di dato

Tipo **Durata**: tipo record con campi

- settimane: Intero ≥ 0 [0..1]
- giorni: intero ≥ 0 [0..1]
- ore: tipo intervallo : 0..24 [0..1]
- minuti: tipo intervallo 0..60

Tipo **Email**: Stringa secondo lo standard

Tipo **Denaro**: Tipo record con campi

- importo: Reale ≥ 0
- valuta: Stringa di 3 caratteri alfabetici maiuscoli

Tipo **Indirizzo**: Tipo record con campi

- via: Stringa
- civico: Stringa
- CAP: Stringa di 5 caratteri numerici

3. Specifica delle operazioni di classe

Operazioni ausiliarie

non_sovraposti(iniz1, fin1, iniz2, fin2) : Boolean

precondizioni:

- $iniz1 < fin1$ e $iniz2 < fin2$

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- $result = True$
se e solo se
 $fin1 \leq iniz2$ or $fin2 \leq iniz1$

in_periodo(iniz1, fin1, iniz2, fin2) : Boolean

precondizioni:

- $iniz1 < fin1$ e $iniz2 < fin2$

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- $result = True$
se e solo se
 $iniz1 \leq iniz2$ and $fin2 \leq fin1$

Specifica della classe Utente

[V.Utente.partecipa.AttivitadiViaggio]

Per ogni u: Utente tale che esiste

- (u, at): u_at, at: Attivita e (v,at): v_at, v: Viaggio

deve esistere

- (u, v): partecipa

viaggi_partecipa() : Viaggio[0..*]

precondizioni: nessuna

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
 - il risultato "result" è definito come segue
 - sia V l'insieme di v: Viaggi tali che (this, v): partecipa
- result = insieme di v: Viaggio in v

attivit _in_viaggio(v: Viaggio) : Attivit  [0..*]

precondizioni:

- esiste un link (this, v): partecipa

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
 - il risultato "result" è definito come segue
 - sia A insieme di tutti at: Attivit  tali che (v, at): v_at e (this, at): ut_at
- result = insieme di at in A

Specifica della classe Viaggio

[V.Viaggio.luogoDestinazione.Uguale.ProssimoLuogoAttivit ]

per ogni v: Viaggio

- sia A l'insieme di tutti at: Attivit  tali che (v, at): v_at
- per ogni copia sp1 e at2 in A tali che
 - sp1: Spostamento,
 - sp1.fine_attivit  ≤ at2.istante_inizio
 - non esiste nessun sp2: Spostamento in A tale che:
 - sp1.fine_attivit  < sp2.istante_inizio
 - sp2.fine_attivit  < at2.istante_inizio
- sia l1: Luogo tale che (sp1, l1): destinazione
- sia l2: Luogo tale che (at2, l2): si_tiene_in

deve essere vero che:

- l1 = l2

[V.Viaggio.Attivit NonSovrapostiNelViaggio]

per ogni v: Viaggio

- sia A l'insieme di tutti at: Attivit  tali che (v, at): v_at
- per ogni at1 e at2 in A deve essere vero che:
non_sovraposti(
at1.istante_iniz,
at1.fine_attivit ,
at2.istante_iniz,
at2.fine_attivit ) = True

[V.Viaggio.minBudget]

per ogni v: Viaggio

- sia A l'insieme di tutti at: Attivit  tali che (v, at): v_at
- sia sum   la somma di tutti at.prezzo.importo con at.prezzo.valuta uguale per at in A

deve essere vero che:

v.budget ≥ sum

[V.Viaggio.MinMaxPartecipanti]

per ogni v: Viaggio deve essere vero che:

- $v.\text{num_min_partecipanti} \leq v.\text{num_max_partecipanti}$

[V.Viaggio.LimitPartecipanti]

per ogni v: Viaggio

- sia U insieme di u:Utenti tali che (u, v): partecipa

deve essere vero che:

- $v.\text{num_min_partecipanti} \leq |U| \leq v.\text{num_max_partecipanti}$

attivit _max_partecipanti() : Attivit 

precondizioni:

- esiste almeno un link (this, at): v_at, at:Attivit 

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
 - il risultato "result" viene definito come segue:
 - sia A insieme dei at:Attivit  tali che (this, at): v_at
 - sia max   il valore massimo di at.partecipanti() per at in A
 - sia at_max: Attivit  in A tale che at_max.partecipanti() = max
- result = at_max

citta_toccate() : Citta [0..*]

precondizioni: nessuna

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
 - il risultato "result" viene definito come segue:
 - sia A l'insieme di at: Attivit  tali che (this, at): v_at
 - sia L l'insieme di lu: Luogo tali che (at, lu): si_tiene_in, at in A
 - sia C l'insieme di cit: Citta tali che (cit, lu): luog_cit, lu in L
- result = insieme di cit: Citta in C

Specifica della classe Attivit 

Operazioni ausiliarie

ha_figlie(): Boolean

precondizioni: nessuna

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- il risultato "result" viene definito come segue:
 - sia F insieme di at: Attivit  tali che (this, at): sotto_attivit 
 - se $|F| = 0$
 - result = False
 - se $|F| > 0$
 - result = True

Vincoli esterni

[V.Attivita.FineDopolnizio]

per ogni at: Attivita deve essere vero che

- $at.istante_inizio < at.fine_attivita$

[V.Attivita.CampiPermessi]

per ogni at: Attivita tale che $at.ha_figlie() = False$

- sia cdp: CampiDurataPermessi definito come segue
 - se at: AttivitaSemplice
 - deve esistere tip: TipoAttivita tale che
 - $(at, tip): at_tip$
 - $(tip, cdp): tip_un$
 - se at: Spostamento
 - deve esistere mez: Mezzo tale che
 - $(at, mezz): spost_mez$
 - $(mez, cdp): mez_ut$

deve essere vero che

- se $cdp.settimane = False$ il campo $at.durata.settimane$ non è valorizzato
- se $cdp.giorni = False$ il campo $at.durata.giorno$ non è valorizzato
- se $cdp.ore = False$ il campo $at.durata.ore$ non è valorizzato

Tra i campi consentiti almeno uno deve essere > 0

durata_minuti() : Intero > 0

precondizioni: nessuna

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- il risultato "resut" viene definito come segue:
 - se $this.ha_figlie = False$
 - $d = this.durata$
 - $result = d.settimane * 10080 + d.giorni * 1440 + d.ore * 60 + d.minuti$
 - se $this.ha_figlie = True$
 - sia F insieme di at: Attivita tali che
 - at è discendente di this
 - $at.ha_figlie = False$
 - $result = \text{somma di tutti } at.durata_minuti() \text{ per ogni at in F}$

fine_attivita() : DataOra

precondizioni: nessuna

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- il risultato "resut" viene definito come segue:
 - se $this.ha_figlie = False$
 - $result = this.istante_iniz + this.durata_minuti()$
 - se $this.ha_figlie = True$
 - sia F insieme di at: Attivita tali che
 - at è discendente di this
 - $at.ha_figlie = False$
 - sia t_max il massimo valore di $at.fine_attivita()$ per at in F
 - $result = t_max$

partecipanti() : Utente[0..*]

precondizioni: nessuna

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- il risultato “result” è definito come segue:
 - se this.ha_figlie = False
 - sia P insieme di u: Utenti tali che (u, this): u_at
 - result = insieme di u in P
 - se this.ha_figlie = True
 - sia F insieme di at: Attività tali che
 - at è discendente di this
 - at.ha_figlie = False
 - sia P insieme di tutti ar.partecipanti() per ogni at in F
 - result = insieme di u in P

Specifica della classe AttivitaSemplice

Vincoli esterni

[V.AttivitaSemplice.SenzaFiglie]

per ogni ats: AttivitaSemplice deve essere vero ats.ha_figlie = False

Specifica della classe Spostamento

Vincoli esterni

[V.Spostamento.LuoghiDiversi]

per ogni sp: Spostamento

- sia l1: Luogo tale che (sp, l1): partenza
- sia l2: Luogo tale che (sp, l2): arrivo

deve essere vero che

- l1 diverso da l2

citta_partenza(): Citta

precondizioni:

- esiste un link (this, lp): partenza, lp:Luogo
- esiste un link (lp, cit): luog_cit, cit: Citta

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- il risultato “result” è definito come segue
result = cit

citta_arrivo(): Citta

precondizioni:

- esiste un link (this, la): partenza, la:Luogo
- esiste un link (la, cit): luog_cit, cit: Citta

postcondizioni:

- operazione non modifica il livello estensionale dei oggetti e link
- il risultato “result” è definito come segue
result = cit

Specifica della classe AttivitaComposta

Vincoli esterni

[V.AttivitaComposta.HaFiglie]

per ogni atc: AttivitaComposta deve essere vero che $atc.ha_figlie = True$

[V.AttivitaComposta.IncludeFigliDiretti]

per ogni atc: AttivitaComposta

- sia F l'insieme di tutti at: Attivita tali che $(atc, at): sotto_attivita$
- per ogni at in F deve essere vero che
 - $in_periodo(atc.istante_inizi, atc.fine_attivita, at.istante_inizi, at.fine_attivita) = True$

[V.AttivitaComposta.noCicli]

versione 1

per ogni $(at[1], at[2], \dots, at[n]): Attivita$, per ogni $n > 0$, tali che

- $(at[1], at[2]): sotto_attivita$
- $(at[2], at[3]): sotto_attivita$
- ...
- $(at[i-1], at[i]): sotto_attivita$
- ...
- $(at[n-1], at[n]): sotto_attivita$

non deve essere $(at[i], at[i-k]): sotto_attivita$ per ogni k in $[0..i-1]$ e i in $[1..n]$

versione 2

per ogni atc: AttivitaComposta non deve esistere nessun $link(atc, atc1): sotto_attivita$, $atc1: AttivitaComposta$ tale che crea il ciclo che riporta $(atc, atc): sotto_attivita$

1.5. Diagramma UML degli Use-Case

uguale

1.6. Specifica delle operazioni di Use-Case

Specifiche di operazioni di use-case Statistiche

Dato un viaggio e un utente che vi partecipa, calcolare il numero di attività del viaggio di tipo "spostamento" alle quali ha preso parte.

num_spostamenti_in_viaggio(u:Utente, v: Viaggio): Intero ≥ 0

precondizioni:

- esiste $link(u, v): partecipa$

postcondizioni:

- operazione non modifica i dati
- Il risultato "result" è così definito

- sia S l'insieme dei sp: Spostamenti tali che
 - (u, sp): u_at
 - (v, sp): v_at
 - sp.ha_figlie() = False

result = | S |

Dato un viaggio, ottenere gli utenti che hanno partecipato al viaggio e che hanno preso parte al maggior numero di attività organizzate in esso

utenti_partecip_max_num_attivita_in_viaggio(v: Viaggio): Utente [1..*]

precondizioni:

- esiste almeno un link (u, v): partecipa, u: Utente
- esiste almeno un link (v, at): v_at, at: Attività

postcondizioni:

- operazione non modifica i dati
- Il risultato "result" è così definito
 - sia U_t l'insieme di tutti u: Utenti tali che (u, v): partecipa
 - sia A l'insieme di tutti at: Attività tali che (v, at): v_at e **at.ha_figlie = False**
 - sia N insieme delle copie (u, n) di valori tali che
 - u in U_t
 - n = la cardinalità dell'insieme dei link (u, at): u_at, at in A
 - sia n_max il valore massimo di valori n in coppie (u, n) in N
 - sia U_{A_max} l'insieme di tutti u delle copie (u, n) in N, il valore di quali n = n_max

result insieme di u: Utenti in U_{A_max}

I responsabili del sistema, data una nazione, devono poter calcolare, per ognuna delle sue regioni, la/e città toccata/e dal maggior numero di viaggi.

citta_con_max_viaggi(naz: Nazione): (Regione, Città) [1..*]

precondizioni:

- deve esistere almeno v: Viaggio tale che (v, at): v_at dove at: Attività tale che
 - esiste un link (at, lu): si_tiene_in, lu: Luogo tale che
 - esiste un link (lu, cit): luog_cit, cit: Città tale che
 - esiste link (cit, reg): cit_reg, reg: Regione tale che
 - esiste un link (reg, naz): reg_naz

postcondizioni:

- operazione non modifica i dati
- il risultato "result" è così definito:
 - sia R insieme delle reg: Regione tale che (reg, naz): reg_naz
 - Sia **M** l'insieme delle coppie (**reg, cit_max**) definite come segue:
 - per ogni reg in R
 - sia C l'insieme di tutte cit: Città tali che (cit, reg): cit_reg
 - sia **NC** insieme delle coppie (**cit, n_cit**) tali che
 - per ogni c in C
 - sia V_cit l'insieme di tutti v: Viaggio tali che esistono at: Attività e lu: Luogo tali che
 - (v, at): v_at
 - **at.ha_figlie = False**
 - (at, lu): si_tiene_in
 - (lu, cit): luog_cit

■ sia n_cit = | V_cit |

- sia n_max il massimo valore n_cit per cit in C
- sia C_max insieme di tutte le cit in C tali che (cit, n_cit) e $n_cit = n_max$
- aggiungi a M tutte le copie (reg, cit) per ogni cit in C_max

result = insieme di coppie (reg, cit_max) in M

Data una città di destinazione, per ognuno dei 12 mesi dell'anno, calcolare il numero di viaggi organizzati in tale mese nell'ultimo anno solare, in tale città

num_viaggi_nel_mese_anno_precedente(cit: Città): (mese: intero in [1..12], intero ≥ 0) [12]

precondizioni:

- deve esistere almeno un link (at, lu) : luogo_cit , at: Attività, lu: Luogo tale che
 - esiste un link (lu, cit) : luogo_cit, cit:Città tale che
 - esiste link (cit, reg) : cit_reg, reg: Regione tale che
 - esiste un link (reg, naz) : reg_naz

postcondizioni:

- operazione non modifica i dati
- il risultato "result" è così definito:
 - sia $y = anno(oggi) - 1$ ultimo anno solare
 - sia M l'insieme di m tali che hanno valore intero da 1 a 12
 - sia N l'insieme di coppie (m, n) tali che
 - per ogni m in M
 - V_m l'insieme di tutti v : Viaggi tali che esistono at: Attività, lu: Luogo che
 - (v, at) : v_at
 - $at.ha_figlie = False$
 - (at, lu) : si_tiene_in
 - (lu, cit) : $luog_cit$
 - $anno(at.istante_inizi) = y$
 - $mese(at.istante_inizi) = m$
 - sia V_m_d insieme di tutti v : Viaggi distinti in V_m
 - $n = |v_m_d|$

result = insieme di coppie (m, n) in N